

Лабораторная работа №5

Вероятностные тесты простоты: Ферма, Соловэй–Штрассен, Миллер–Рабин

Кюнкриков Даниил Саналович

2026-03-18

Информация

Вводная часть

Содержание исследования

Выполнение работы

Итоговый слайд

1. Информация

Кюнкриков Даниил Саналович

студент уч. группы НПМд-01-24

Российский университет дружбы народов (РУДН)

1132249574@pfur.ru

репозиторий: https://github.com/DanzanK/2025-2026_math-sec/tree/main

2. Вводная часть

Проверка чисел на простоту — базовая задача криптографии (генерация ключей, параметры протоколов).

Для больших n используются быстрые **вероятностные** тесты, которые либо находят свидетеля составности, либо дают вывод «вероятно простое» с контролируемой ошибкой.

Объект исследования: алгоритмы проверки чисел на простоту.

Предмет исследования: тест Ферма;

символ Якоби и тест Соловья–Штрассена;

тест Миллера–Рабина;

программная реализация и демонстрация на примерах.

Цель: реализовать вероятностные алгоритмы проверки чисел на простоту на Python.

Задачи: реализовать тест Ферма;

реализовать вычисление символа Якоби;

реализовать тест Соловья–Штрассена;

реализовать тест Миллера–Рабина;

провести тестирование и представить результаты.

Язык программирования: **Python**.

Методика: многократные проверки для случайных оснований a (раунды), количество раундов — параметр t .

Вывод: «составное» при нахождении свидетеля, иначе — «вероятно простое».

3. Содержание исследования



Параметры:

n — проверяемое число,

t — число раундов,

a — случайное основание (свидетель).

На каждом раунде выбирается a и проверяется критерий; нарушение критерия означает, что n **точно составное**.

Для простого n (и $\gcd(a, n) = 1$) выполняется:

$$a^{n-1} \equiv 1 \pmod{n}.$$

Если равенство нарушено, найден свидетель составности.

4. Выполнение работы

```
# тест Ферма
def fermat_test_single(n: int) -> str:
    a = random.randint(2, n - 2)
    r = pow(a, n - 1, n)
    return "possible prime" if r == 1 else "composite"

def run_feramn_test(n: int, t: int) -> str:
    if n < 5:
        return "input должен быть >= 5"
    if n % 2 == 0:
        return "composite even"
    for _ in range(t):
        if fermat_test_single(n) == "composite":
            return "composite"
    return "possible prime"
```

Рисунок 1: Программная реализация теста Ферма

Символ Якоби $(a/n) \in \{-1, 0, 1\}$ вычисляется по правилам вынесения степеней 2 и квадратичной взаимности.

Критерий Эйлера для простого n :

$$a^{(n-1)/2} \equiv (a/n) \pmod{n}.$$

Оценка ошибки для составного n : не более 2^{-t} .

Реализация: Якоби и Соловэй–Штрассен

```
# вычисление символа Якоби
def jacobi(a:int, n: int) -> int:
    if n <= 0 or n % 2 == 0:
        raise ValueError("n должно быть нечетным и положительным")
    a %= n
    result = 1
    while a != 0:
        while a % 2 == 0:
            a //= 2
            r = n % 8
            if r == 3 or r == 5:
                result = -result
        a, n = n, a

    if a % 4 == 3 and n % 4 == 3:
        result = -result
    a %= n

    return result if n == 1 else 0
```

Рисунок 2: Программная реализация расчета символа Якоби

```
# тест Соловэя-Штрассена
def solov_strass_test_single(n: int) -> str:
    a = random.randint(2, n-2)
    r = pow(a, (n-1) // 2, n)
    if r != 1 and r != n-1:
        return "composite"

    s = jacobi(a,n)
    return "possible prime" if r == (s+n) % n else "composite"

def run_solov_strass_test(n: int, t: int) -> str:
    if n < 5:
        return "input должен быть >=5"
    if n%2 == 0:
        return "composite even"
    for _ in range(t):
        if solov_strass_test_single(n) == "composite":
            return "composite"
    return "possible prime"
```

Рисунок 3: Программная реализация теста Соловэя–Штрассена

Представим:

$$n - 1 = 2^s \cdot d, \quad d \text{ нечётное.}$$

Вычисляем $y_0 = a^d \bmod n$ и далее цепочку квадратов $y_{k+1} = y_k^2 \bmod n$.

Если ни разу не получено значение $-1 \bmod n$, то найден свидетель составности. Оценка ошибки: не более 4^{-t} .

```
# тест Миллера-Рабина

def miller_rabin_test_single(n: int) -> str:
    s = 0
    r = n-1
    while r%2 ==0:
        s += 1
        r //= 2
    a = random.randint(2, n-2)
    y = pow(a,r,n)

    if y != 1 and y != n-1:
        j = 1
        while j <= s - 1 and y != n - 1:
            y = pow(y,2,n)
            if y == 1:
                return "composite"
            j += 1
        if y != n - 1:
            return "composite"
        return "possible prime"

def run_miller_rabin_test(n: int, t: int) -> str:
    if n < 5:
        return "input должен быть >=5"
    if n%2 ==0:
        return "composite even"
    for _ in range(t):
        if miller_rabin_test_single(n) == "composite":
            return "composite"
    return " possible prime"
```

Рисунок 4: Программная реализация теста Миллера-Рабина

```
введи n: 131
введи t: 20
тест Ферма:           possible prime
тест Соловья-Штрассена: possible prime
тест Миллера-Рабина:  possible prime
```

Рисунок 5: Результат работы программы для выбранных значений n и числа раундов t .

5. Итоговый слайд



Реализованы вероятностные тесты простоты: Ферма, Соловэй–Штрассен, Миллер–Рабин.

Реализовано вычисление символа Якоби.

Выполнена демонстрация работы программы; получены результаты для различных n и параметра t .